RIDE-SHARING APPLICATION

CASE STUDY

Submitted by

S.Keerthana (2023BCSE07AED476)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (CSE-GENERAL)

Under the Supervision of

Dr. Rajkumar N

ALLIANCE SCHOOL OF ADVANCED COMPUTING ALLIANCE UNIVERSITY, BENGALURU

February - 2026

Table of Contents

1. Introduction

The development of a ride-sharing application comes with specific challenges in software engineering. Unlike standard taxi booking systems, ride-sharing platforms work in a changing environment affected by traffic, varying demand, safety rules, and digital payment laws. This case study looks at how a clear Software Requirements Document (SRD) and an effective Requirements Management (RM) process assist in managing changing business models, safety policies, and technology updates, all while ensuring reliability, scalability, and legal compliance**.**

2. The Challenge: Transportation Environment

The ride-sharing industry works in a constantly changing environment

Scenario A: Government Transport Regulations

Suppose a city government enacts a new rule that requires panic buttons and ride tracking logs for passenger safety. The application must quickly update its functional requirements to include emergency response integration.

Scenario B: Surge Pricing Restrictions

Surge Pricing Restrictions A regulatory authority may limit surge pricing during emergencies or natural disasters. The fare calculation algorithm must be changed to meet these rules.

Core Issues:

- **Scope Creep:** Scope creep caused by new features such as ride pooling and subscription rides.
- **Compliance Risk:** Compliance risk if safety laws are broken.
- **Conflict Management:** Conflict between usability, like quick booking, and security, such as identity verification.

3. Stakeholder Analysis in Requirements Engineering

Identifying stakeholders is essential for ride-sharing systems.

- Passengers care about safety, affordability, and convenience.
- Drivers care about fair earnings, navigation accuracy, and incentives.
- Regulatory authorities concentrate on transport safety laws and taxes.
- The technical team focuses on scalability, uptime, and performance.
- Business stakeholders emphasize revenue, market growth, and competition.

Conflict Resolution: Passengers want one-click booking. Meanwhile, regulators require driver background checks and ride OTP validation. The RM process balances both.

4. The Regulatory Requirements Engineering Cycle

To manage changing regulations, the standard RE cycle must be adjusted:

4.1 Elicitation: Market and Regulatory Intelligence

Requirements for the ride-sharing application are gathered through a structured process that sources information from multiple places to ensure completeness and accuracy. We conduct passenger surveys to understand user expectations around safety, affordability, ease of booking, real-time tracking, payment preferences, and overall experience. Driver interviews offer insights into operational challenges like route optimization, fair fare calculation, incentive structures, fuel costs, app usability, and safety concerns during trips.

4.2 Analysis & Negotiation

After collecting requirements, the team carefully examines them to understand their technical feasibility, cost, and possible conflicts with other system requirements. At this stage, the development team considers whether each requirement can be realistically implemented within the available time, budget, and technology limits. They also identify and resolve conflicts through discussions and negotiations among stakeholders.

For example, while accurate real-time GPS tracking can improve user experience and safety, it may also lead to higher battery consumption and increased mobile data usage. In these cases, the team looks at trade-offs to find a balance between tracking accuracy and energy efficiency, possibly by optimizing update intervals. They use a risk-based analysis to prioritize critical requirements, making sure that safety features like emergency SOS buttons and secure authentication are more important than non-essential aesthetic changes. This structured evaluation helps create a system that is practical, reliable, and meets both business and safety goals.

4.3 Validation: Business and Legal Approval

Before implementation begins, the proposed requirements go through formal validation to check their correctness and feasibility. The legal team reviews them to ensure they comply with transport regulations and data protection laws. Meanwhile, business stakeholders assess and approve the related costs and budget implications. Additionally, prototype testing takes place to clarify functionality, collect early feedback, and confirm that everyone understands the requirements before development starts.

5. Role of Requirements Management Process

The Requirements Management (RM) process is essential for identifying, documenting, analysing, tracking, and controlling requirements during the software development lifecycle. It offers a clear way to manage changes and keep track of the relationship between requirements and design or code components. This process also ensures that the project meets business and regulatory goals. By regularly checking requirement updates and assessing their effects on scope, cost, and schedule.

5.1 Impact Analysis and Traceability

In the Requirements Management process, each requirement is linked to specific design modules, code components, and test cases. Traceability helps with impact analysis when changes happen. For instance, if surge pricing regulations are updated, developers can quickly trace the affected requirement to the specific pricing engine module in the design and code. This allows them to make necessary changes without affecting unrelated parts of the system

5.2 The Change Control Board (CCB)

The Change Control Board (CCB) is a formal decision-making group that reviews and approves changes to requirements. It typically includes the Project Manager, Legal Advisor, Lead Developer, and Business Analyst. The CCB looks at the cost impact, timeline impact, and overall risk level of each proposed change before giving approval. This process ensures that changes are managed and fit with project goals.

5.3 Baseline Management

Baseline Management means defining a fixed and approved set of requirements for a specific release, like "Version 1.0 Ride Booking Module." Once a baseline is set, changes are carefully managed to prevent any disruption to ongoing development. If new laws or requirements come up during development, they are planned for a future release instead of changing the current baseline. This approach helps maintain project stability and progress.

6. Integrating Requirements into Agile Development

Functional requirements spell out what the system actually does.

6.1 Core System Features Here's what the system handles:

Agile development offers the flexibility needed to address changing business requirements, regulatory updates, and evolving user expectations in a ride-sharing application. Instead of strict planning, features are developed in short iterations, or sprints. This allows for continuous feedback and improvement.

- **Regulatory User Stories:** "As a passenger, I want a panic button so that I feel safe during rides," clearly defines both the functionality and the purpose behind it. This method keeps development focused on users while addressing legal and safety issues.
- **Definition of Done (DoD):** It ensures quality and compliance before any feature is marked as complete. A feature is finished only after it has passed functional testing, security validation, and compliance approval. This stops incomplete or non-compliant features from being released.

- **Continuous Compliance:** Compliance scanners are part of the development workflow. Automated test scripts run after each code update to check that critical components, such as payment processing and GPS tracking, continue to work correctly. This method helps catch errors early, maintain system stability, and ensure that new updates do not disrupt existing functionality.

7. Ensuring Functional and Non-Functional Requirements

To keep the application on track despite changes, we need to include specific engineering tasks directly in the process.

7.1 Managing Functional Requirements:

Functional requirements outline what the ride-sharing system needs to do to fulfill user and business needs. These include essential features like user registration with OTP verification for secure access, options for booking and cancelling rides, real-time GPS tracking to monitor trips, fare calculation and digital payment processing, an efficient driver assignment method, a ratings and feedback system, and an emergency SOS feature for passenger safety. Each functional requirement must be clearly documented, uniquely identified, and traceable to design components, code modules, and test cases to ensure proper implementation and easy analysis of impacts when changes happen.

7.2 Managing Non-Functional Requirements (NFRs)

Non-functional requirements describe how well the system should perform and outline the quality traits of the application. For the ride-sharing platform, this includes performance requirements like keeping a response time under three seconds, security steps such as encrypted transactions and secure authentication methods, high availability of at least 99.9% uptime, scalability to handle more than 10,000 concurrent users, reliability to avoid data loss during payment processing, and usability standards that require rides to be bookable in three simple steps. We conduct continuous security audits and use performance testing scripts to monitor these traits and maintain compliance with system standards.

8. Validation and Verification Criteria

To make sure that all requirements are met, we conduct a thorough validation process. We perform functional testing to check key workflows like ride booking, cancellation, and payment processing. We carry out security testing, such as penetration testing and vulnerability scanning, to find and remove possible threats. Finally, User Acceptance Testing (UAT) involves real drivers and passengers testing the system to ensure it meets real-world expectations and business needs before it goes live.

9. Measuring Success: Key Performance Indicators (KPIs)

The success of the requirements management process and overall system performance is measured using key performance indicators (KPIs). The Requirement Volatility Index tracks how often requirements change during development. This shows the stability and clarity in planning. Traceability Coverage measures the percentage of implemented features that are linked to documented requirements. This ensures completeness and accountability. The Defect Leakage Rate identifies the number of bugs found after deployment. This reflects the effectiveness of testing. System Uptime Percentage evaluates application availability and reliability. Customer Satisfaction Ratings assess user experience and overall service quality.

10. Conclusion

Developing a ride-sharing app requires constant adjustments to changing rules, new technologies, and shifting business strategies. A well-organized Software Requirements Document (SRD), backed by a solid Requirements Management (RM) process, lays the groundwork for managing these changes effectively. It keeps functional and non-functional requirements clearly defined, which helps with planning and implementation. The process also highlights the importance of proper validation and verification to ensure that all requirements are implemented and tested accurately. By using systematic change management practices, updates are controlled and evaluated for impact before they are put into action, which reduces project risks.